Mark	3/11

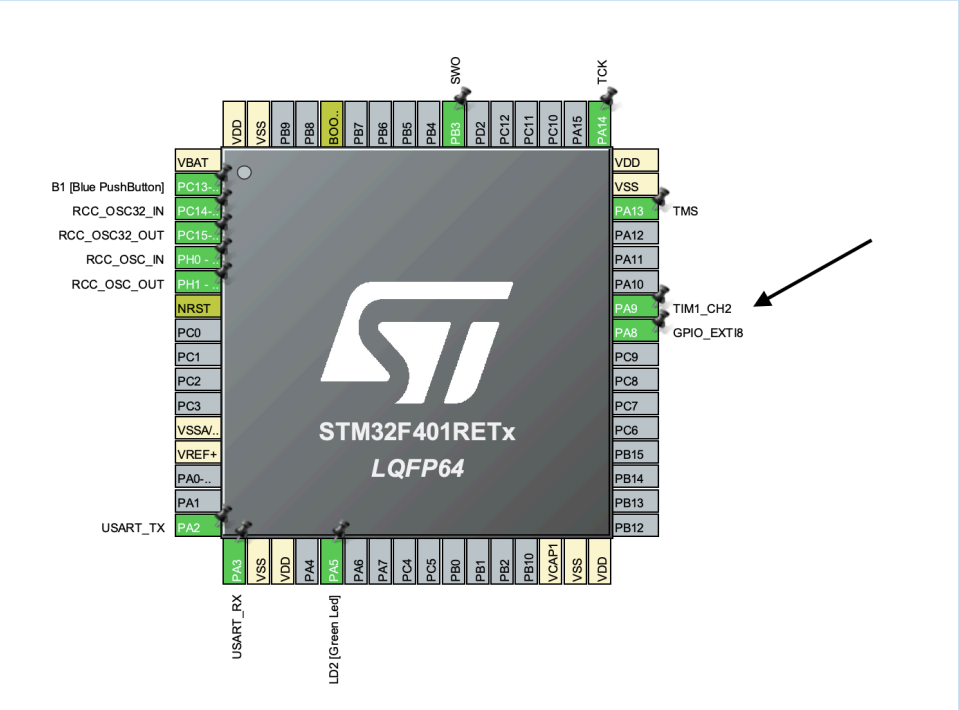
Team name:	B9		
Homework number:	Homework 03		
Due date:	07/10/2025		
Contribution:	**No**	**Partial**	**Full**
Pica Mirko			✅
Prendin Christian			✅
Pignataro Gabriele			✅
Moazenkakhki Mostafa			✅
Yarrabbotula Sunandha			✅
Notes:			

Project name	Play a song		
Not done	**Partially done (major problems)**	**Partially done (minor problems)**	**Completed**
			✅

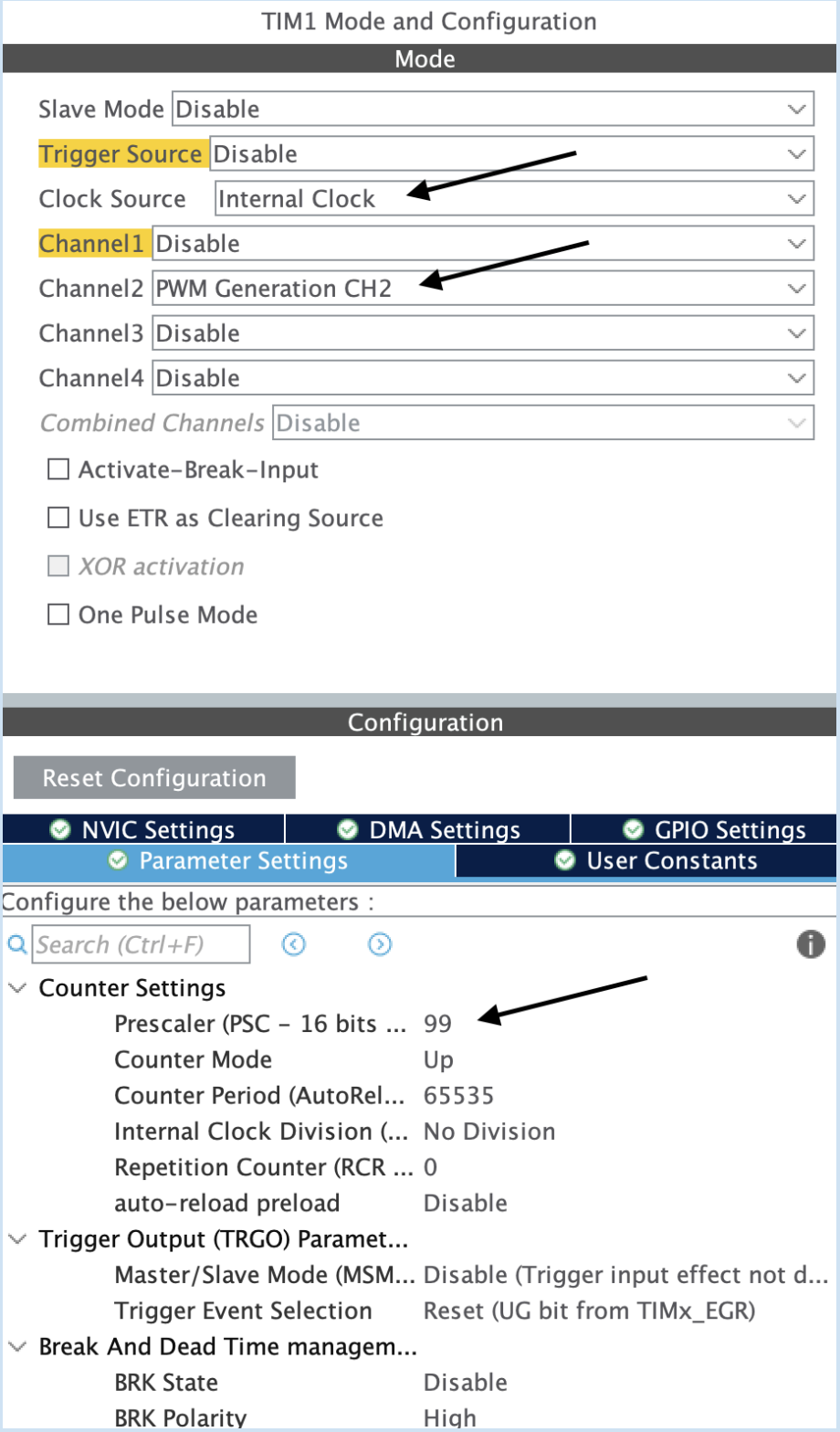
We successfully completed the project by doing the following.

Part 1. Conclude the project 2C "Play a song" using the HAL_Delay function.
Using the "Hands-on Lab Schematics.pdf" and "Nucleo Schematics.pdf" we found that the speaker is connected to pin PA9, and from the past homework we know that the microphone is connected to pin PA8.

So we hop into the IDE and configure them to our necessities. PA9 as TIM1_CH2 as we will use the timer to do PWM generation and PA8 as GPIO_EXTI8.



We need to configure the timer TIM1 to do PWM generation on CH2. For parameters we just set the prescaler to 100 since we will change ARR in the code to obtain the different notes.



For the microphone it is the same configuration as last time, so we enable the EXTI interrupt and we set the pin with rising edge trigger detection, this way we generate an interrupt every time there is a rising edge, so that every time we make a sound to the microphone.

Finally we go to the “main.c” where we define a struct for our tone, made of the note and its duration, we define all notes ARR using the calculations done with the provided file “M 05 - Musical note.xlsx” and the TEMPO, used to modify the speed of the song.

```c
/* Private typedef -----------------------
/* USER CODE BEGIN PTD */
typedef struct {
    uint16_t tone;
    uint16_t duration;
} Note;
/* USER CODE END PTD */

/* Private define ------------------------
/* USER CODE BEGIN PD */
#define DO4 3205
#define DOD4 3031
#define RE4 2856
#define RED4 2700
#define MI4 2544
#define FA4 2406
#define FAD4 2269
#define SOL4 2142
#define SOLD4 2023
#define LA4 1908
#define LAD4 1802
#define SI4 1699

#define TEMPO 80
/* USER CODE END PD */
```

We define our song as an array of tones (duration one corresponds to a chrome in the pentagram) and also define a variable to check if the song is already playing.

```
/* USER CODE BEGIN PV */
const Note song[24] = {
        {SOL4, 7},
        {LA4, 1},
        {SOL4,4},
        {FA4,4},
        {MI4,4},
        {FA4,4},
        {SOL4,8},
        {RE4,4},
        {MI4,4},
        {FA4,8},
        {MI4,4},
        {FA4,4},
        {SOL4,8},
        {SOL4,7},
        {LA4,1},
        {SOL4,4},
        {FA4,4},
        {MI4,4},
        {FA4,4},
        {SOL4,8},
        {RE4,8},
        {SOL4,8},
        {MI4,4},
        {DO4,12}
};
volatile uint8_t isPlaying = 0;
/* USER CODE END PV */
```

We add the following functions.

```c
void setNote(uint32_t note) {

	TIM_ClockConfigTypeDef sClockSourceConfig = {0};
	TIM_MasterConfigTypeDef sMasterConfig = {0};
	TIM_OC_InitTypeDef sConfigOC = {0};
	TIM_BreakDeadTimeConfigTypeDef sBreakDeadTimeConfig = {0};

	htim1.Instance = TIM1;
	htim1.Init.Prescaler = 99;
	htim1.Init.CounterMode = TIM_COUNTERMODE_UP;

	htim1.Init.Period = note - 1;

	htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
	htim1.Init.RepetitionCounter = 0;
	htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
	if (HAL_TIM_Base_Init(&htim1) != HAL_OK)
	{
	  Error_Handler();
	}
	sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
	if (HAL_TIM_ConfigClockSource(&htim1, &sClockSourceConfig) != HAL_OK)
	{
	  Error_Handler();
	}
	if (HAL_TIM_PWM_Init(&htim1) != HAL_OK)
	{
	  Error_Handler();
	}
	sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
	sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
	if (HAL_TIMEx_MasterConfigSynchronization(&htim1, &sMasterConfig) != HAL_OK)
	{
	  Error_Handler();
	}
	sConfigOC.OCMode = TIM_OCMODE_PWM1;

	sConfigOC.Pulse = note/2 - 1;

	sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
	sConfigOC.OCNPolarity = TIM_OCNPOLARITY_HIGH;
	sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
	sConfigOC.OCIdleState = TIM_OCIDLESTATE_RESET;
	sConfigOC.OCNIdleState = TIM_OCNIDLESTATE_RESET;
	if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_2) != HAL_OK)
	{
	  Error_Handler();
	}
	sBreakDeadTimeConfig.OffStateRunMode = TIM_OSSR_DISABLE;
	sBreakDeadTimeConfig.OffStateIDLEMode = TIM_OSSI_DISABLE;
	sBreakDeadTimeConfig.LockLevel = TIM_LOCKLEVEL_OFF;
	sBreakDeadTimeConfig.DeadTime = 0;
	sBreakDeadTimeConfig.BreakState = TIM_BREAK_DISABLE;
	sBreakDeadTimeConfig.BreakPolarity = TIM_BREAKPOLARITY_HIGH;
	sBreakDeadTimeConfig.AutomaticOutput = TIM_AUTOMATICOUTPUT_DISABLE;
	if (HAL_TIMEx_ConfigBreakDeadTime(&htim1, &sBreakDeadTimeConfig) != HAL_OK)
	{
	  Error_Handler();
	}

	HAL_TIM_MspPostInit(&htim1);
}
```

This is basically the init function for Timer1 and is used to modify the tone played by the speaker.

```c
void playSong() {
    int s = sizeof(song)/sizeof(song[0]);
    for (size_t i = 0; i < s; i++) {
        setNote(song[i].tone);
        HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
        HAL_Delay(song[i].duration * TEMPO);
        HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_2);
    }
}

void HAL_GPIO_EXTI_Callback(uint16_t pin) {
    if (pin == GPIO_PIN_8 && !isPlaying) {
        isPlaying = 1;
        playSong();
        isPlaying = 0;
    }
}
/* USER CODE END 0 */
```

This block contains the core logic to play the song. Every time the interrupt is triggered, the request comes from the microphone pin and the song is not already playing, we start playing the song. To play the song we iterate over the array and for every tone we set the correct tone, start the PWM, wait for the duration using HAL_Delay and stop the PWM.
We have tested it and it works.

Part 2. Try to do the project 2C "Play a song" again without using the HAL_Delay function. Hint: use a timer interrupt.
Following the hint, this time we didn't use a delay for the duration of the note but a timer.
So we first enabled TIM2 and just set the prescaler to 8400 and enabled its interrupt.

In the "main.c" file we modify and add the following functions.

```c
void setTimer(uint32_t duration) {
      TIM_ClockConfigTypeDef sClockSourceConfig = {0};
      TIM_MasterConfigTypeDef sMasterConfig = {0};

      htim2.Instance = TIM2;
      htim2.Init.Prescaler = 8399;
      htim2.Init.CounterMode = TIM_COUNTERMODE_UP;

      // Multiply by 10 so that EIGHTH value is in ms
      htim2.Init.Period = 10 * TEMPO * duration - 1;

      htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
      htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
      if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
      {
        Error_Handler();
      }
      sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
      if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
      {
        Error_Handler();
      }
      sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
      sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
      if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
      {
        Error_Handler();
      }
}

void startNote(Note n) {
    setNote(n.note);
    setTimer(n.duration);
    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
    HAL_TIM_Base_Start_IT(&htim2);
}

void HAL_GPIO_EXTI_Callback(uint16_t pin) {
    if (pin == GPIO_PIN_8 && !isPlaying) {
        isPlaying = 1;
        startNote(song[currentNote]);
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *tim) {
    if (tim->Instance == TIM2 && isPlaying) {
        HAL_TIM_Base_Stop_IT(&htim2);
        HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_2);

        currentNote++;
        if (currentNote < songLen) {
            startNote(song[currentNote]);
        } else {
            currentNote = 0;
            isPlaying = 0;
        }
    }
}
/* USER CODE END 0 */
```

Which means that every time the interrupt is triggered, the request comes from the microphone pin and the song is not already playing, we start the first note. In the sartNote function we set the correct tone, start the PWM, and set the second timer to the duration of the note (we needed to multiply for 10 to get the correct ratio). So when the timer elapses it stops the PWM, and if there are other notes to play it goes to the next one. Also, we have introduced the variables *currentNote* to keep track of the note to be played and *songLen* which is just the number of notes in the song. We have tested it and it works.	
Professor comments:	